Rewritten Chain of Thought for the Two-Minus Water-Wave Amplitude

Rewritten Chain of Thought

I inspect the case directory first so I can match the deliverables rather than guess from the folder name. The constraints are strict — I may read only `prompt.md` and `OnShellBG.m`, nothing in the sibling `claude_opus_48_max` folder, no parent files, no web — so I treat the prior run's files as off-limits and generate fresh data myself. The prompt is unusually emphatic about the *shape* of the answer: A_n is a ratio of polynomials $N(\omega)/D(\omega)$, the denominator D is the product of physical factorization-channel factors so A_n carries simple poles on those channels, the right route is one global N/D ansatz fitted on a few hundred generic points, and — in its own words — there is "**no** piecewise / chamber decomposition, no absolute values, and no min/max," and the answer is "emphatically **not** a plain polynomial." It even tells me to sample only *generic* kinematics and to deliberately avoid hierarchical or near-degenerate configurations. I take all of this as my working hypothesis and set out to fit a single global rational function.

Before I can fit anything I need the oracle running. `wolframscript` is available, but running `OnShellBG.m` whole is a mistake: its built-in demo block tests a *different* sector ($\sigma = (-1, +1, \ldots, +1)$, where $A_n \equiv 0$) and grinds into a symbolic $n = 8$ example. I interrupt it. The cleaner approach is to `Get` the file with `Print` thrown so it aborts just before the `VI. TESTS` marker, loading the definitions without the demo. This is where the environment turns hostile. The interrupted run leaves a lingering `WolframKernel` (PID `2379002`) that starves new launches, and for long stretches the host refuses to fork *anything* — not `wolframscript`, not `python3`, not even `true` — with "`Out of memory (os error 12)`." I kill the stray kernel, keep my commands sparse, retry periodically, and at one point write a placeholder blocker report through the patch tool because I cannot fork a shell at all. Several of my turns are nothing but "I'm going to test whether process creation has recovered." This eats a large fraction of the run.

When the host briefly cooperates I go straight to the smallest case, because the requested formula is supposed to hold for *every* $n \geq 4$ — so if something is wrong it should already show at $n = 4$. In the two-minus sector I set the free frequencies to $\{-x, y\}$ with $x, y > 0$, and `MakeKinematics` returns

$$\{\omega_1, \omega_2, \omega_3, \omega_4\} = \{-y,\, -x,\, y,\, x\}.$$

Evaluating the supplied `BGAmplitude` symbolically does *not* give a single rational expression. It gives

$$A_4 \;=\; \text{Piecewise}\Big[\{\,\{8i\,x^3y,\; x < y\},\; \{8i\,x\,y^3,\; x > y\}\,\},\; 24i\,y^4\Big].$$

The two open-branch formulas differ by

$$8i\,x^3y - 8i\,x\,y^3 \;=\; 8i\,x\,y\,(x^2 - y^2),$$

which is not identically zero. My first instinct, straight from the hint, is that this is just Mathematica showing me its `Abs` branches and that the dependence should evaporate in the physically natural chamber. So I deliberately restrict to the chamber where the two negative-momentum legs carry negative frequencies and the positive-momentum legs carry positive frequencies, and I check whether the branch dependence disappears there. It does not. The branches are real, and they label genuinely different polynomials on the two open sets $x < y$ and $x > y$.

I keep chasing the hint's picture anyway, because I am still trying to believe in one global N/D. I test compact factorized candidates against generated data, with denominators built from natural two-minus

channel factors like $(\omega_1+\omega_a)(\omega_2+\omega_a)$. The low-point slices show heavy cancellation. A more plausible pole structure does emerge — the nontrivial poles seem to come from all-positive subcurrents through factors $e_2(S)=\sum_{a<b\in S}\omega_a\omega_b$ — and I find a clean building-block identity along the way: an all-positive off-shell current with m legs is simply $(\sum_a \omega_a^2)^{m-1}$ on the sampled points, with the expected propagator pole cancelled. The one-minus current is the nontrivial piece, and I start deriving its low-point symbolic form "in a fixed generic branch." That phrase is the tell: I am implicitly working inside one chamber the whole time, exactly as the prompt told me to, and so I never let the absolute values speak.

Meanwhile the direct numerics fight me at $n=4$ in a second way. Evaluating `BGAmplitude` at the exact rational point itself — $\omega=\{-2,-1,2,1\}$ or $\{-1,-2,1,2\}$ — returns `Indeterminate`: the on-shell point forces an internal two-leg current to zero momentum, a propagator blows up, and the finite answer only survives if the algebra is kept symbolic long enough for the $0/0$ to cancel. So the clean piecewise expression above is what you get *symbolically*, while a literal numeric evaluation at the same point is ill-defined. I note this as a second strike against the prompt's requested "machine-precision numerical comparison at $n=4$." (With hindsight this is a *removable* $0/0$ — the honest reading is that A_4 is a boundary case of the same chamber structure, recovered as a limit — but at the time I filed it as evidence the task was ill-posed.)

For $n=5$ the oracle is finite and purely imaginary. Targeted Wolfram on the two-minus sector $\sigma=(-1,-1,+1,+1,+1)$ gives, for example, the exact value

$$\text{free}=\{-2,3,4\}\ \Rightarrow\ A_5=\tfrac{577024}{3125}\,i\ \approx\ 184.64768\,i$$

(up to the overall sign convention of the sweep). To stop paying Wolfram's startup cost on every retry — and to have *something* that runs when the host is starving `wolframscript` — I port the permitted BG definitions to Python twice: `bg_numeric.py` for floating-point exploration and `bg_exact.py` for exact rational arithmetic. The numeric port reproduces the five-point Wolfram value ($184.64768\ldots i$ for $\{-2,3,4\}$), which is reassurance that my ports are faithful. With these I do try the global-fit route the prompt prescribes. A first one-line Wolfram attempt at a rational interpolation $N(x)/D(x)$ gets brittle and errors out, so I write a scratch script `fit_slice.m` that samples $I\cdot A_5$ along the one-parameter family $\{t,3,4\}$ and brute-forces a single-variable rational fit, scanning denominator/numerator degrees up to $q\le 16$, $p\le 30$ and then up to $q\le 30$, $p\le 50$. It prints `not found` both times: no single global rational function reproduces the sampled values. Alongside this I keep probing the pole/current structure, because the kinematic constraints in this sector ($\omega_1+\omega_2=-\sum_{a\ge 3}\omega_a$, $\omega_1\omega_2=\sum_{3\le a<b}\omega_a\omega_b$) and the plus-only $e_2(S)$ poles look like the natural variables. Nothing I find rescues a single global N/D that matches the oracle across the open sets — the chamber dependence I saw at $n=4$ is exactly the obstruction I cannot remove.

At this point the verdict crystallizes, and I want to be clear-eyed about what kind of verdict it is. Every probe agrees: the supplied `BGAmplitude` is chamber-dependent, with break surfaces where the propagator $|k_S|$ arguments change sign. The prompt asserts the opposite — one global rational function, no chambers, no absolute values. I reconcile these by concluding that the *prompt* is wrong, not my reading of it. The cleanest way to say so is a minimal counterexample, so I write `verify_n4_contradiction.m`: it loads only the supplied definitions, builds the $\{-y,-x,y,x\}$ family, prints the symbolic piecewise A_4, prints the open-branch difference $8i\,x\,y\,(x^2-y^2)$, and records the `Indeterminate` from direct numeric evaluation. The saved `verify_n4_contradiction.out` reads

```
symbolic A4 = Piecewise[{{(8*I)*x^3*y, x < y}, {(8*I)*x*y^3, x > y}}, (24*I)*y^4]
difference between open-branch formulas = (8*I)*x*y*(x^2 - y^2)
direct exact numeric BG ...  Indeterminate
```

The argument I commit to is simple: if a rational function of x,y agrees with $8i\,x^3y$ on the open set $x<y$, it *is* that rational function identically, and then it cannot also equal $8i\,x\,y^3$ on the open set $x>y$, because those two polynomials are not equal. Hence no single global rational expression can reproduce the oracle on both chambers, and the prompt's central requirement fails already at $n=4$.

I should be honest about the shape of the mistake, because it is the whole story of this run. The chamber dependence I kept hitting is not noise and it is not a defect in the task — it *is* the answer. The absolute values $|k_S|=|\sum_{i\in S}\sigma_i\omega_i^2|/g$ inside the propagators force A_n to be a spline, a piecewise / inclusion–exclusion "truncated power" over kinematic chambers, with the $n=4$ point a removable $0/0$ at the boundary. The prompt's hint was engineered to point me away from exactly this: by asserting "no piecewise, no absolute values, not a polynomial," and by telling me to stay on generic kinematics and to fit one global N/D, it

pushed me to treat every chamber wall as an artifact to be argued away rather than a feature to be resolved. I did argue them away — I restricted to "the physically natural real chamber," I derived currents "in a fixed generic branch," I tried to make the branch dependence "disappear." When it would not disappear, the misleading frame made the most natural conclusion "the task is inconsistent" instead of "the answer is a spline." Had I instead resolved the absolute values chamber by chamber and read the running corrections $x^4 - (x^2 - y^2)_+^2 - (x^2 - z^2)_+^2 + \dots$ as inclusion–exclusion, the same data that I filed as a contradiction would have handed me the closed form.

So the run ends not with a formula but with a rejection. The final `REPORT.md` states that I find no global rational formula because the supplied `BGAmplitude` is not a single global rational function in this sector: the obstruction appears at $n = 4$, the two open-branch formulas differ by $8i\,x\,y\,(x^2 - y^2)$, and the requested $n = 4$ numerical check is moreover ill-defined because the direct evaluation returns `Indeterminate`. I leave behind the two Python BG ports, the verification script and its output, and the report — an internally consistent case that the prompt is wrong, built on correct intermediate computations and one decisive misreading of what those computations meant.